

TDD Manifesto

Test Driven Development in every home

MENU



A clean test

Clean code requires clean tests

A master craftsman (e.g. carpenter) finishes any part of a product with the same care as the front, regardless of the fact that some of the parts won't be that visible. Software engineering is also about craftsmanship where we should **write all parts of the software with equal care**, whether it is production or test code.

Writing tests is a part of our craftsmanship. We can only have clean code if we have clean tests. We are aware that the cleanliness of a test depends on the programming language, testing framework, toolset, and the problem domain itself. We believe that tests should fulfill a list of objective characteristics, resulting in clean, readable, and maintainable tests.

A clean test reads like a story. All information necessary to follow the story must be contained in the test itself. We should not expect to search for some other content to understand the story. A clean test should contain all the information necessary to understand what is being tested.

Hypothesis

Let's imagine that we are building a webshop application. The users are able to search for products, selecting them, collecting the products in a shopping cart, and finally, purchasing them. In the context of the application we have a use-case with the following specifications:

- Given a shopping cart that has products with a total grand 50
- When the total grand is calculated
- Then it returns 50 as total grand



Non-clean test implementation

```
Test()  
{  
    var sc = new ShoppingCart();  
    sc.Add(new Product("id1", 10, "product 1", ProductCategory.Book));  
    sc.Add(new Product("id2", 15, "product 2", ProductCategory.Electronic));  
    sc.Add(new Product("id3", 25, "product 3", ProductCategory.Hardware));  
    var p = sc.Calc();  
    Assert.True(p == 50);  
}
```

Characteristics of a clean test

Descriptive test name

The name of a test should reveal the exact test case, including the system under test. It should specify the requirement of the test case as accurately as possible. The main purpose of a good test name is that if a test fails, then we should be able to retrieve the broken functionality from the test name.

Two popular naming conventions:

GivenWhenThen

This approach is based on naming convention developed as part of Behavior-Driven Development (BDD). By using this convention we split our test into three parts such as precondition, execution of system under test and expected behaviour.

Examples:

- GivenUserIsNotLoggedIn_whenUserLogsIn_thenUserIsLoggedInSuccessfully
- GivenInvalidCreditCardData_whenOrderIsPurchased_thenReturnsCreditCardIsInvalidValidationError
- GivenAccountBalanceIs1000Euro_when100EuroIsDeposited_thenAccountBalanceIs1100Euro

ShouldWhen

ShouldWhen template is an easy to read and widely used naming template. The name first reveals the expected behavior, then the precondition of the given test case.

Examples:

- ShouldHaveUserLoggedIn_whenUserLogsIn
- ShouldReturnCreditCardIsInvalidValidationError_whenTheOrderIsPurchasedWithInvalidCreditCardData
- ShouldIncreasebalanceWith100Euro_when100EuroDepositIsMadeToTheAccount

Note that these are just conventions and not rigid rules, and there can be situations where other test naming conventions could also efficiently reveal the intent of the test.

Meaningful namings

Using clean, meaningful, and intention-revealing namings in the tests is as important as using clean namings in the production code. Therefore we should use clean namings in areas such as:

- software element names (f.e.: class, function, variable names)
- preparation scripts
- execution of system under test
- assertions of expected behaviors

Tips for naming your software elements:

- follow naming conventions
- don't pollute names with technical details
- use functional namings relating to the business domain
- use named constants for magic numbers/strings
- use pronounceable namings
- do not use custom abbreviations
- be explicit instead of implicit
- reveal intent with well-named functions instead of using comments

Structured with the Arrange-Act-Assert (AAA) pattern

The Arrange-Act-Assert pattern is a descriptive and intention-revealing way to structure test cases. It prescribes an order of operations:

- The **Arrange** section should contain the set-up logic for the tests. Here the objects are initialized and prepared for the execution of the system under test (SUT)
- The **Act** section invokes the system we are about to test. It can be for example calling a function, calling a REST API, or interacting with some we component
- The **Assert** section verifies that the action of the SUT behaves as expected. For example, here we check the return value of a method, the final state of the SUT and its collaborators, the methods the SUT called on them, or possible expected exceptions and error results.

In practice, each part is separated by a new line from each other, but there can be complex scenarios when the each part can be annotated using comments such as `//Arrange`, `//Act` or `//Assert` (where “`//`” is for example a syntax for creating a comment in the code).

Follows the F.I.R.S.T principle

The F.I.R.S.T. principle is an acronym containing 5 important characteristics of a clean test.

Fast

A test should be fast and efficient. As the application grows, we will have more and more tests. Therefore running the as fast as possible is essential. The faster our tests are, the shorter the feedback loop is, and the

faster we can detect the failures in our application.

Independent

A test should not depend on the state of any other tests or on external services. It should be standalone to be able to execute it individually and efficiently.

Repeatable

A test should be repeatable in any environment. Each test should set up its own data and should not depend on any external factors to run its test. A clean test should be deterministic that always results in the same tests succeeding.

Self-validating

A test should validate itself whether the test execution is passed or failed. The self-validating test can avoid the need to do an evaluation manually by us.

Thorough

The tests we write:

- should cover all happy paths
- should cover edge/corner/boundary cases
- should cover negative test cases
- should cover security and illegal issues

Asserts one behavior

A test should verify a single behavior. A single behavior can contain one or multiple lines of assertions in the code. A test should be coupled to a functional behavior and not to a technical action or change in the code.

Uses meaningful test data

Tests are examples of code usages. They are executable documentation for the use cases. They should use meaningful test data relating to the business domain, resulting in readable, useable, and real-life examples. Therefore revealing domain knowledge by using meaningful test data is essential to produce clean tests.

Hide irrelevant data for the test

Do not pollute your tests with irrelevant test data. Such information just increases the cognitive mental load, resulting in bloated tests. Instead, hide irrelevant data by using test data builders.



Clean test implementation



```
GivenShoppingCartHasProductsWithTotalPrice50_whenCalculateTotalPrice_thenTotalPriceIs50()  
{  
    var shoppingCart = new ShoppingCart();  
    shoppingCart.Add(CreateProduct().WithPrice(10));  
    shoppingCart.Add(CreateProduct().WithPrice(15));  
    shoppingCart.Add(CreateProduct().WithPrice(25));  
  
    var totalPrice = shoppingCart.CalculateTotalPrice();  
  
    Assert.That(totalPrice).IsEqualTo(50);  
}
```

